Age como um Agente de Segurança Web3 e Blockchain Security Researcher.

O teu objetivo é fazer uma pesquisa exaustiva na internet sobre os principais bugs, exploits, hacks e vulnerabilidades que afectaram protocolos de agregadores de DEX (DEX Aggregators) em EVM.
Instrução de Pesquisa:
Pesquisa activamente em fontes como:
Rekt.news (base de dados de hacks DeFi)
Immunefi (bug bounties e post-mortems)
Chainalysis / Certik / Trail of Bits (relatórios de auditoria)
GitHub (CVEs e issues públicos em repositórios de aggregators)
Twitter/X (threads de security researchers como @samczsun, @pcaversaccio, @bytes032)
Medium / Mirror (post-mortems técnicos de protocolos)
O Que Pesquisar:
Hacks históricos confirmados em DEX Aggregators:
Nome do protocolo, data, valor perdido (USD), vector de ataque.
Exemplos conhecidos: 1inch (router exploit), Paraswap, KyberSwap, Odos, etc.
Para cada hack: resume o vector de ataque em 2-3 linhas técnicas.
Vectores de ataque mais comuns em aggregators (por categoria):
Reentrancy em routers multi-hop
Price manipulation / Oracle manipulation (Chainlink, TWAP)
Approval abuse (infinite approve exploits, Permit2 issues)
Callback exploits em tokens maliciosos (ERC-777, fee-on-transfer)
Slippage manipulation / Sandwich attacks
Frontend hijacking (DNS spoofing, malicious calldata injection)
Calldata manipulation (quando o frontend passa calldata directamente para o router)
Fee manipulation (manipulação de parâmetros de fee off-chain)
Cross-chain bridge exploits em aggregators multi-chain
Bugs em Smart Contracts de Routers/Aggregators:
Problemas conhecidos com SwapRouter02 (Uniswap), Augustus (Paraswap), Settlement (CoW Protocol), routers Odos, etc.
Vulnerabilidades em contratos de fee collection.
Vulnerabilidades específicas de Infraestrutura Off-Chain:
APIs de aggregators com rate limiting bypassado.
Manipulação de quotes off-chain antes de submissão on-chain.
Ataques a event indexers / bases de dados de protocolos.
Formato de Output Esperado:
A) Tabela de Hacks Confirmados

Para cada incidente:
| Protocolo | Data | Valor Perdido | Vector Principal | Mitigação Aplicada |
B) Top 10 Vectores de Ataque mais Comuns
Lista ordenada por frequência/impacto, com:
Descrição técnica breve do vector
Exemplo real (se existir)
Como mitigar (best practice)
C) Vulnerabilidades Específicas Relevantes para um Agregador com:
7 fontes de liquidez (1inch, 0x, Velora, Odos, KyberSwap, CoW Protocol, Uniswap V3 directo)
Fee off-chain calculada com validação on-chain mínima
Permit2 + EIP-2612 + Exact Approve como sistema de approvals
Chainlink price feeds para validação de preço
Stack: Next.js, wagmi v2, viem, Ethereum mainnet
Para esta secção, foca-te nas vulnerabilidades que directamente se aplicam a esta arquitectura específica.
D) Recomendações de Segurança Prioritárias
Lista de acções concretas ordenadas por prioridade (crítica / alta / média), com referência ao ficheiro/componente específico onde deve ser aplicada.
Restrições:
Cita sempre a fonte (URL ou nome do relatório) para cada hack ou vulnerabilidade.
Distingue claramente entre vulnerabilidades confirmadas/exploradas e teóricas/potenciais.
Foca-te em incidentes de 2020 até hoje (mais relevantes para a arquitectura actual).
Se não encontrares informação suficiente numa fonte, indica-o e passa para a seguinte.

Agora tenho dados suficientes para construir um relatório exaustivo. Vou compilar toda a informação.

Web3 Security Research Report: DEX Aggregator Exploits & Vulnerabilities (EVM)

Scope: Incidentes confirmados 2020–2026 + vectores de ataque relevantes para a tua arquitectura específica (7 fontes de liquidez, Permit2/EIP-2612, Chainlink, Next.js/wagmi v2/viem).

A) Tabela de Hacks Confirmados em DEX Aggregators

Protocolo	Data	Valor Perdido	Vector Principal	Mitigação Aplicada	Fonte
CoW Swap (GPv2Settlement)	Fev 2023	~$166K–$181K	Arbitrary call execution no SwapGuard (solver externo); DAI approval drenado via `transferFrom` arbitrário	Revogação de approvals; slashing do solver; redesign de validação de solver	[1] [2]

Protocolo	Data	Valor Perdido	Vector Principal	Mitigação Aplicada	Fonte
Unizen DEX Aggregator	Mar 2024	~$2.1M	Unsafe external call introduzida em upgrade do contrato de agregação; aprovações existentes drenadas via `transferFrom` não validado	Pausa do contrato (15 min após detecção); bounty on-chain ao atacante	[3] [4]
ParaSwap Augustus V6	Mar 2024	~$24K directo ($2.7M white-hatted)	Critical allowance-drain bug no Augustus V6 recém-lançado; aprovações infinitas exploráveis antes de qualquer swap	Auto-destruct do V6; white-hat de $2.7M; revert para V5; revogação de 386 endereços	[5] [6]
KyberSwap Elastic	Nov 2023	~$48M	Tick manipulation + double-counting de liquidez; erro de rounding de precisão (<`0.00000000001`%) em concentrated liquidity pool explorado via flash loan	Desactivação das pools afectadas; investigação de recuperação de fundos	[7] [8]
1inch Fusion V1 Resolvers	Mar 2025	~$5M	Vulnerabilidade em contratos de resolvers usando implementação Fusion v1 desactualizada; fundos de resolvers (não de end-users) drenados	Patch de emergência; alerta a resolvers para audit e upgrade imediato	[9]
1inch Router v4–v6 (Rescue Function)	2025 (descoberto Jul 2025)	~$520K	Design flaw: funções públicas de callback aceitam pools swap definidas pelo utilizador; tokens mis-sent drenáveis por qualquer actor via spoofed transactions	Patch à rescue function; verificação de legitimidade de transacções	[10] [11]
SwapNet / Matcha Meta	Jan 2026	~$16.8M	Arbitrary-call com input validation insuficiente; users com infinite approval fora do One-Time Approval framework do 0x expostos a `transferFrom` de atacante	Desactivação dos contratos SwapNet; urge revogação de approvals directas	[12] [13]

B) Top 10 Vectores de Ataque em DEX Aggregators

1. Arbitrary External Call / Unchecked Calldata Injection

Frequência: CRÍTICA — vector #1 confirmado em múltiplos hacks.

O router/aggregator aceita calldata arbitrário do utilizador ou de um solver externo e executa low-level calls sem validar o destino ou a função chamada. O atacante constrói calldata que invoca `transferFrom` em tokens já aprovados, drenando fundos de utilizadores.[12] [3]

- **Exemplo real:** Unizen (Mar 2024) — unsafe external call em contrato upgradado; SwapNet/Matcha Meta (Jan 2026) — arbitrary-call com input validation insuficiente[3] [12]
- **Mitigação:** Whitelist estrita de endereços destino e function selectors permitidos; nunca executar low-level `.call()` com calldata 100% controlado pelo utilizador; allowlist de DEX adapters validados on-chain

2. Approval Abuse via Infinite Approve + Contract Upgrade

Frequência: CRÍTICA — presente em 3+ hacks confirmados.

Utilizadores concedem infinite approval a um contrato de aggregator. Quando esse contrato é upgradado (ou um bug é introduzido), o novo código pode drenar todas as aprovações existentes sem novo consentimento do utilizador.[5] [13]

- **Exemplo real:** ParaSwap Augustus V6 (Mar 2024) — 386 endereços com approvals infinitas expostos imediatamente após deploy do V6 bugado; SwapNet via Matcha Meta (Jan 2026) — users com One-Time Approval desactivado perderam $16.8M[13] [5]
- **Mitigação:** Forçar One-Time Approval (Permit2 com expiração); nunca pedir infinite approve para contratos upgradáveis; smart contract imutável separado para custody de approvals

3. Permit2 / EIP-2612 Signature Phishing

Frequência: ALTA — crescente com adopção de Permit2.

Atacantes induzem utilizadores a assinar mensagens EIP-712 que parecem inócuas mas concedem allowances via `permit()` ou Permit2 `PermitTransferFrom`. A assinatura offline não requer gas, tornando o vector quase invisível.[14]

- **Exemplo real:** Phishing campaigns no Ethereum mainnet explorando Permit2 da Uniswap — utilizadores enganados a assinar approvals mascaradas como acções normais[14]
- **Mitigação:** Validar `spender`, `amount`, e `deadline` antes de assinar qualquer Permit2 message; UI deve mostrar decoded permit details prominentemente; usar `PERMIT2_PERMIT_TYPEHASH` com deadlines curtos

4. Solver/Interaction Arbitrary Execution (CoW-style)

Frequência: ALTA — específico a arquitecturas intent-based/solver.

Em protocolos com modelo de solver (CoW Protocol, 1inch Fusion), um solver malicioso ou comprometido pode submeter interactions arbitrárias no contexto do settlement contract, aprovando tokens para contratos maliciosos.[1]

- **Exemplo real:** CoW Swap (Fev 2023) — Barter Solver aprova SwapGuard bugado; atacante executa arbitrary call para drenar DAI do GPv2Settlement[1]
- **Mitigação:** Validação rigorosa de interaction data pelo settlement contract; allowlist de contratos com que o settlement pode interagir; slashing económico de solvers maliciosos

5. Flash Loan + Tick/Liquidity Manipulation (Concentrated Liquidity)

Frequência: ALTA — impacto muito elevado ($48M num único incidente).

O atacante usa flash loan para manipular o preço de uma pool de concentrated liquidity para uma zona sem liquidez, ganhando controlo total dos cálculos. Explora erros de rounding em operações cross-tick para double-count liquidez e drenar a pool.[8]

- **Exemplo real:** KyberSwap Elastic (Nov 2023) — $48M drenados via precisão de rounding $< 10^{-11}\%$ em `updateLiquidityAndCrossTick`[7] [8]
- **Mitigação:** Invariant checks pós-swap; limites de tick manipulation detection; circuit breakers on-chain por pool

6. Sandwich Attack / MEV Slippage Exploitation

Frequência: ALTA — endémica em qualquer aggregator.

O atacante monitoriza o mempool, detecta um swap de elevado valor, executa front-run a subir o preço, deixa o trade da vítima executar no pior preço possível dentro do slippage tolerance, e faz back-run para realizar o lucro.[15]

- **Exemplo real:** Documentado academicamente e em produção em todos os aggregators EVM públicos; cross-chain sandwich attacks são uma variante emergente[15]
- **Mitigação:** Deadlines estritos de transacção; slippage defaults baixos (≤0.5%); integração com private mempools (Flashbots Protect, MEV Blocker); CoW Protocol como alternativa para MEV resistance

7. Oracle / Price Feed Manipulation

Frequência: ALTA — especialmente em aggregators que usam TWAP on-chain.

O atacante manipula o preço spot de uma pool AMM através de flash loan para falsear o TWAP ou o Chainlink fallback, permitindo swaps a preços artificiais ou explorar lógica de routing que

depende de price comparisons.[8]

- **Exemplo real:** KyberSwap (indirectamente — o flash loan foi usado não para oracle mas para manipulação de estado da pool, conceito relacionado)[8]
- **Mitigação:** Chainlink com circuit breakers (heartbeat + deviation threshold); nunca usar spot price como oracle; TWAP com janelas mínimas de 30min para decisões críticas

8. Fee-on-Transfer / Rebase Token Callback Exploits

Frequência: MÉDIA — mas critical para aggregators multi-token.

ERC-777 tokens com callbacks `tokensReceived`/`tokensToSend` permitem reentrância durante transfers. Fee-on-transfer tokens causam discrepâncias entre `amountIn` esperado e real, podendo explorar lógica que assume `balanceOf(after) - balanceOf(before) == amountIn`.[16]

- **Exemplo real:** Clober DEX reentrancy via callback (2024) — $500K[16]
- **Mitigação:** Checks-effects-interactions pattern estrito; usar `balanceOf` pre/post para calcular amounts reais; blocklist de ERC-777 tokens em routers críticos; ReentrancyGuard em todos os entry points

9. Frontend Hijacking / DNS Spoofing + Malicious Calldata

Frequência: MÉDIA — crescente com sofisticação de ataques.

O atacante compromete o DNS ou injeta código no frontend (CDN poisoning, supply chain attack em npm packages), alterando o calldata passado ao router para desviar fundos ou alterar os parâmetros de `minAmountOut`, `recipient`, ou `fee` antes de o utilizador assinar [confirmado como vector teórico com precedentes em outros protocolos].

- **Mitigação:** Content Security Policy estrita; SRI hashes para todos os scripts; validação on-chain de `recipient == msg.sender`; mostrar calldata decoded na wallet UI antes de assinar

10. Off-Chain Fee Manipulation + Insuficiente Validação On-Chain

Frequência: MÉDIA — específico a aggregators com fee calculada off-chain.

Quando a fee é calculada no backend/API e passada como parâmetro no calldata, sem validação on-chain rigorosa, um atacante pode interceptar ou manipular o parâmetro `fee` (via MITM, request replay, ou manipulação de quote) para pagar zero fees ou inflacionar o `minAmountOut` [vector teórico com alta relevância para arquitectura com fee off-chain + validação on-chain mínima].[12]

- **Mitigação:** Assinar o objecto de quote completo (incluindo fee, deadline, recipient) com chave privada do backend; validar assinatura on-chain no router antes de executar; nunca confiar em parâmetros não assinados passados pelo frontend

C) Vulnerabilidades Específicas para a Tua Arquitectura

Arquitectura: 7 fontes de liquidez (1inch, 0x, Velora, Odos, KyberSwap, CoW Protocol, Uniswap V3 directo) + fee off-chain + Permit2/EIP-2612/Exact Approve + Chainlink + Next.js/wagmi v2/viem

C.1 — ⊠ CRÍTICO: Fee Off-Chain sem Assinatura Verificável On-Chain

A tua arquitectura calcula a fee no backend e passa-a como parâmetro no calldata. Se o contrato router não verificar uma assinatura criptográfica do backend sobre `(amountIn, amountOut, fee, deadline, recipient, chainId)`, qualquer utilizador pode modificar o calldata via intercepção de API, remix de transaction, ou frontend comprometido, pagando zero fee ou alterando o destinatário. Este é exactamente o vector do SwapNet/Matcha Meta onde parâmetros não validados foram explorados.[13] [12]

- **Ficheiro afectado:** `contracts/AggregatorRouter.sol` (função de swap/execute) + `pages/api/quote.ts` (backend de geração de quote)
- **Fix:** Implementar EIP-712 signature pelo backend sobre o quote completo; o contrato verifica `ecrecover` antes de processar qualquer swap

C.2 — ⊠ CRÍTICO: Calldata Passthrough Arbitrário para 7 Adapters Externos

Ao rotear para 7 fontes de liquidez diferentes, o teu aggregator provavelmente executa `.call()` nos contratos externos (1inch, 0x, Odos, etc.) com calldata gerado off-chain. Se este calldata não for validado (function selector + destino em allowlist), replica o vector do Unizen (unsafe external call) e do SwapNet (arbitrary-call via insufficient input validation).[3] [12]

- **Ficheiro afectado:** `contracts/adapters/*.sol` (cada adapter) + `contracts/AggregatorRouter.sol`
- **Fix:** Whitelist estrita de `(address, bytes4 selector)` permitidos por adapter; revert se destino ou selector não estiver na allowlist

C.3 — ⊠ CRÍTICO: Permit2 com Deadline Longo + Signature Phishing

A tua UI pede aos utilizadores para assinar Permit2 `PermitTransferFrom`. Se o `deadline` for longo (ou indefinido) e o `spender` for o teu router (upgradável ou com vulnerabilidades latentes), uma assinatura phishada ou um bug futuro no router pode drenar fundos muito depois da transacção original. Este é o vector confirmado de phishing Permit2 em Ethereum mainnet.[14]

- **Ficheiro afectado:** `hooks/usePermit2.ts` (wagmi v2) + frontend de assinatura
- **Fix:** Deadline máximo de 15–30 minutos em todos os Permit2 messages; mostrar `spender`, `amount`, e `deadline` decoded na UI antes de assinar; nunca usar `PERMIT_AMOUNT_EXPIRATION = MAX_UINT`

C.4 — ⊠ ALTA: Infinite Approve para Múltiplos Contratos Externos

A arquitectura suporta Exact Approve como fallback. Se um utilizador der infinite approve ao teu router (ou aos adapters de 1inch/0x/Odos), um bug em qualquer um desses contratos cria exposição permanente — exactamente o que aconteceu com ParaSwap V6 (386 endereços) e SwapNet ($16.8M).[5] [13]

- **Ficheiro afectado:** `components/ApprovalButton.tsx` + `hooks/useTokenApproval.ts`
- **Fix:** Default para Exact Approve (`amountIn` exacto) em vez de infinite; Permit2 como opção preferida com expiração curta; UI que mostra claramente o scope da aprovação

C.5 — ⊠ ALTA: Chainlink Price Feed — Stale Data e Manipulação de Heartbeat

Se usas Chainlink para validar preços on-chain mas não verificas `updatedAt` e `answeredInRound`, um feed stale (Chainlink heartbeat missed) pode deixar passar swaps a preços desactualizados. Em cenários de volatilidade extrema ou congestionamento da rede, este gap pode ser explorado [vector teórico com alta relevância].

- **Ficheiro afectado:** `contracts/PriceValidator.sol`
- **Fix:**

```
(, int256 price,, uint256 updatedAt,) = priceFeed.latestRoundData();
require(block.timestamp - updatedAt <= MAX_STALENESS, "Stale price");
require(price > 0, "Invalid price");
```

C.6 — ⊠ ALTA: Cross-Aggregator Slippage — minAmountOut Manipulável

O teu sistema agrega quotes de 7 fontes e passa `minAmountOut` ao router. Se este valor for calculado off-chain com base num quote que pode ser stale por 1–2 segundos, e o contrato não tiver validação de slippage on-chain independente (usando o Chainlink feed como referência de sanidade), o utilizador fica exposto a sandwich attacks ou execução a preços muito piores que o esperado.[15]

- **Ficheiro afectado:** `contracts/AggregatorRouter.sol` (parâmetro `minAmountOut`)
- **Fix:** Calcular `minAmountOut` floor no contrato usando Chainlink como sanity check; revert se o output final estiver >X% abaixo do preço Chainlink (ex: `require(amountOut >= chainlinkPrice * amountIn * (1 - MAX_SLIPPAGE))`)

C.7 — ⊠ MÉDIA: wagmi v2 / viem — Deserialização de Calldata no Frontend

Em Next.js com wagmi v2 e viem, o calldata é construído no frontend (ou recebido da API) e passado directamente para `writeContract` / `sendTransaction`. Se houver uma vulnerabilidade XSS no Next.js app ou um supply chain attack num package npm, o calldata pode ser substituído em memória antes de ser assinado — o vector de frontend hijacking [vector teórico com alta relevância].

- **Ficheiro afectado:** `app/swap/page.tsx` + `wagmi.config.ts`

- **Fix:** Validar o calldata recebido da API contra o schema esperado (Zod/viem ABI encode verification) antes de passar ao wagmi; implementar `simulateContract` (viem) antes de cada `writeContract` para verificar o output esperado

D) Recomendações de Segurança Prioritárias

⊠ Prioridade Crítica

1. **Implementar EIP-712 backend signature para todos os quotes** — O backend assina `(tokenIn, tokenOut, amountIn, minAmountOut, fee, deadline, recipient, chainId)` com chave privada; o router verifica on-chain via `ecrecover`. Afecta: `contracts/AggregatorRouter.sol` + `pages/api/quote.ts`[12]
2. **Whitelist estrita de adapter addresses e function selectors** — Manter uma allowlist imutável (ou governada por timelock) de `(address, bytes4)` válidos para cada uma das 7 fontes. Revert em qualquer chamada fora da whitelist. Afecta: `contracts/adapters/*.sol`[4] [3]
3. **Eliminar infinite approvals — forçar Permit2 com deadline ≤30min** — Remover qualquer UI flow que solicite infinite approve. Usar `PERMIT2_PERMIT_TYPEHASH` com `deadline = block.timestamp + 1800`. Afecta: `hooks/useTokenApproval.ts`, `components/ApprovalButton.tsx`[5] [13]
4. **Validação on-chain de Chainlink com staleness check e circuit breaker** — Verificar `updatedAt`, `answeredInRound`, e `price > 0` em todas as chamadas. Adicionar circuit breaker que pausa o router se o feed estiver stale >1h. Afecta: `contracts/PriceValidator.sol` [vector C.5 acima]

⊠ Prioridade Alta

5. **Sanity check de `minAmountOut` usando Chainlink on-chain** — Calcular no contrato o floor price aceitável com base no feed Chainlink e revert se o output for >5% abaixo. Afecta: `contracts/AggregatorRouter.sol`[15]
6. **ReentrancyGuard em todos os entry points do router** — Especialmente crítico dado que o router chama 7 contratos externos diferentes. Usar `nonReentrant` do OpenZeppelin em todas as funções públicas `swap()`, `execute()`, `batchSwap()`. Afecta: `contracts/AggregatorRouter.sol`[16]
7. **Simulação de transacção antes de submissão (viem `simulateContract`)** — Em wagmi v2, sempre chamar `publicClient.simulateContract()` antes de `writeContract`. Revert esperado on-chain vai prevenir a maioria dos exploits de calldata manipulado. Afecta: `hooks/useSwap.ts` [vector C.7]
8. **Auditoria de segurança antes de qualquer upgrade de contrato** — Unizen perdeu $2.1M exactamente porque upgradou o contrato sem auditoria da nova versão. Todo o upgrade deve ter auditoria independente + período de timelock mínimo de 48h. Afecta: deploy pipeline[3]

Prioridade Média

9. **Content Security Policy (CSP) + Subresource Integrity (SRI) no Next.js** — Configurar `next.config.js` com headers CSP estritos; adicionar SRI hashes para todos os scripts externos. Protege contra frontend hijacking e supply chain attacks [vector C.7]
10. **Monitoring on-chain com alertas de anomalia** — Implementar monitorização via Tenderly Alerts ou Forta Network para detectar: volume anormal por wallet, drenagem de approvals em série, calls para addresses fora da whitelist. Resposta em <15min (Unizen demorou esse tempo para pausar)[4]
11. **Programa de bug bounty no Immunefi** — Protocolos auditados que também têm bug bounty ativo têm maior probabilidade de detecção responsável de vulnerabilidades antes de exploração. Custo do bounty é negligenciável face ao risco ($16.8M no caso SwapNet)[13]

> **Nota de Âmbito:** Todos os hacks marcados na Tabela A são incidentes confirmados com fontes verificáveis. As vulnerabilidades nas secções C e D marcadas como "vector teórico" são baseadas em padrões de ataque confirmados em protocolos similares, aplicadas à tua arquitectura específica — não foram confirmadas como exploradas no teu sistema.

⁂

1. https://cow.fi/learn/cow-swap-solver-exploit-post-mortem
2. https://dn.institute/attacks/posts/2023-02-07-CoW-Swap/
3. https://www.halborn.com/blog/post/explained-the-unizen-hack-march-2024
4. https://rivanorth.com/blog/hack-explained-unizen
5. https://cointelegraph.com/news/paraswap-hack-augustus-v6-contract-vulnerability
6. https://www.htx.com/feed/news/914368/
7. https://blog.kyberswap.com/post-mortem-kyberswap-elastic-exploit/
8. https://www.halborn.com/blog/post/explained-the-kyberswap-hack-november-2023
9. https://www.tradingview.com/news/cointelegraph:b5d53305d094b:0-1inch-resolver-suffers-5m-hack-due-to-smart-contract-vulnerability/
10. https://www.ainvest.com/news/carbontec-finds-520-000-vulnerability-1inch-router-2507/
11. https://cryptonews.net/news/security/31265486/
12. https://www.mexc.com/news/580155
13. https://www.binance.com/en/square/post/35598626223161
14. https://blog.ueex.com/crypto-hacks/ethereum-chain-uniswap-permit2-malicious-signature/
15. https://arxiv.org/html/2511.15245v1
16. https://rekt.news/cloberdex-rekt
17. https://www.rekt.news
18. https://rekt.news/balancer-rekt2
19. https://rekt.news/zero-to-lend

20. https://rekt.news/unizen-rekt
21. https://www.investing.com/news/cryptocurrency-news/rekt-database-reveals-17-billion-has-been-lost-to-defi-exploits-2604865
22. https://beincrypto.com/defi-exploits-1-7b-rekt-database/
23. https://www.auditone.io/blog-posts/kyberswap-exploit-a-comprehensive-breakdown
24. https://docs.de.fi/audits/rekt-database
25. https://blog.solidityscan.com/kyberswap-hack-analysis-25e25f2e4a7b
26. https://tr.tradingview.com/news/cointelegraph:ff2716d27094b:0-paraswap-evades-hack-targeting-augustus-v6-contract-vulnerability/
27. https://www.binance.com/en/square/post/2024-03-20-paraswap-discovers-serious-vulnerability-affecting-augustus-v6-contract-users-5642202529122
28. https://x.com/CertiK/status/1770421845755392204
29. https://github.com/rotki/rotki/blob/develop/docs/changelog.rst
30. https://firewall.firm.in/mirai-botnet-variant-exploits-four-faith-router-vulnerability-for-ddos-attacks/
31. https://cryptorank.io/news/feed/a7666-paraswap-thwarts-augustus-v6-vulnerability
32. https://routersecurity.org/bugs.php
33. https://tech.yahoo.com/cybersecurity/articles/hackers-infecting-older-asus-routers-190352013.html
34. https://br.tradingview.com/news/cointelegraph:186756a0c094b:0-paraswap-begins-returning-crypto-after-critical-smart-contract-bug/
35. https://www.cryptika.com/hackers-exploiting-cisco-ios-xe-vulnerability-in-the-wild-to-deploy-badcandy-web-shell/
36. https://www.cs.cit.tum.de/fileadmin/w00cfj/sebis/thesis/230915_Lladrovci_Thesis.pdf
37. https://blog.solidityscan.com/cow-swap-hack-analysis-arbitrary-callable-swapguard-6a6ee3de346f
38. https://www.binance.com/ar/square/post/203814
39. https://forklog.com/en/cow-swap-decentralised-exchange-loses-166000-in-hack/
40. https://cryptopotato.com/defi-platform-cow-protocol-loses-over-550-bnb-in-contract-exploit/
41. https://www.binance.com/ar-AE/square/post/203814
42. https://fc24.ifca.ai/preproceedings/155.pdf
43. https://www.linkedin.com/pulse/continuing-sandwich-attack-irony-crypto-scott-germaise-s89de
44. https://www.iosco.org/library/pubdocs/pdf/ioscopd754.pdf